

Yes—for a personal feature branch, rebasing onto `origin/main` is usually the cleaner practice, especially before opening or updating a pull request.

Bash

```
git fetch origin
git rebase origin/main
```

This moves your feature commits on top of the latest `main`:

Before:

```
A---B---C  origin/main
      \
        D---E  feature
```

After rebase:

```
A---B---C---D'---E'  feature
```

The history stays linear, and your PR is easier to review.

Use rebase when:

- The feature branch is mainly yours.
- Your commits have not been widely shared.
- You want a clean, linear history.
- You are updating the branch before merging a PR.

After rebasing, if the branch was already pushed, you will usually need:

Bash

```
git push --force-with-lease
```

Use `--force-with-lease`, not plain `--force`, because it protects you from accidentally overwriting someone else's newer remote commits.

Avoid rebasing when:

- Multiple people are actively working on the same feature branch.
- Other branches are based on your feature branch.
- You do not want to rewrite published commit history.

In those cases, merge instead:

Bash

```
git fetch origin  
git merge origin/main
```

A practical workflow for your own feature branch is:

Bash

```
git status  
git fetch origin  
git rebase origin/main
```

If conflicts occur:

Bash

```
# Fix the conflicted files  
git add <resolved-files>  
git rebase --continue
```

To cancel the rebase:

Bash

```
git rebase --abort
```

Then push:

Bash

```
git push --force-with-lease
```

My general recommendation:

- **Your private feature branch:** rebase.
- **A shared feature branch:** merge.
- **Main or other protected branches:** never rebase published history.

Also, prefer the explicit two-command version over:

Bash

```
git pull --rebase origin main
```

Both can work, but `fetch` followed by `rebase` makes it clearer exactly what Git is doing.

